Neural Computational Graph SC Decoder for 2-User MAC Polar Codes

Master's Thesis in Electrical Engineering

March 2026

Abstract

We present a neural successive cancellation (SC) decoder for two-user multiple access channel (MAC) polar codes that replaces the analytical tree operations of the computational graph framework with learned neural modules. The decoder operates on the $O(N \log N)$ computational graph structure introduced by Ren et al. (2025) for monotone chain decoding paths, preserving the tree navigation and scheduling logic while replacing the three core operations—`calcLeft`, `calcRight`, and `calcParent`—with weight-shared multilayer perceptrons (MLPs). We propose two variants: a *Soft-Bit Bridge* version (27,764 parameters) that retains the analytical circular convolution within the `calcParent` operation for differentiability, and a *Pure Neural* version (38,500 parameters) that eliminates all analytical operations via a GRU-gated residual architecture trained through knowledge distillation. Both versions achieve block error rates (BLER) competitive with or superior to analytical SC decoding across code lengths $N = 8$ to 1024, with the Soft-Bit Bridge version achieving a $2\times$ BLER improvement at $N = 1024$. The neural decoder has complexity $O(md \cdot N \log N)$ where m and d are fixed architectural parameters, making it independent of the channel alphabet size and memory—a critical advantage over analytical SC decoding which scales as $O(|\mathcal{S}|^3 N \log N)$ for channels with memory.

Contents

1 Introduction **3**

2 Background: Polar Codes for the MAC **3**
2.1 Two-User MAC Channel Model . . . 3
2.2 Polar Encoding . . . 4
2.3 Monotone Chain Paths and the Capacity Region . . . 4
2.4 The Computational Graph Decoder . . . 5

3 The CalcParent Problem **6**
3.1 Why Class B Needs CalcParent . . . 6
3.2 Frequency of CalcParent Calls . . . 6
3.3 Why Naive Neural Approaches Fail . . . 6

4 Architecture: Neural Computational Graph Decoder **7**
4.1 Overall Architecture . . . 7
4.2 Neural Modules . . . 7
4.2.1 EmbeddingZ (48 parameters) . . . 7
4.2.2 NeuralCalcLeft (8,336 parameters) . . . 7
4.2.3 NeuralCalcRight (8,336 parameters) . . . 8
4.2.4 Emb2Logits (5,508 parameters) . . . 8
4.2.5 Logits2Emb (5,520 parameters) . . . 8
4.2.6 No-Info Embedding (16 parameters) . . . 8
4.2.7 NeuralCalcParent — Pure Neural Version (10,736 parameters) . . . 9

5 The Soft-Bit Bridge **9**
5.1 Architecture . . . 9
5.2 Why It Works . . . 10
5.3 Differentiability . . . 10
5.4 Temperature Parameter . . . 10
5.5 Limitation . . . 10

6 The Pure Neural CalcParent **10**
6.1 Motivation . . . 10
6.2 GRU-Gated Architecture . . . 10
6.3 Knowledge Distillation Training . . . 11

7 One Decoding Step **11**
7.1 Step 1: Navigate to Target Leaf . . . 12
7.2 Step 2: Save Bottom-Up Embedding . . . 12
7.3 Step 3: Compute Top-Down Message . . . 12
7.4 Step 4: Combine Top-Down and Bottom-Up . . . 12
7.5 Step 5: Make Decision . . . 12
7.6 Step 6: Update Leaf to Partially Deterministic State . . . 13

8 Training **13**
8.1 Teacher Forcing and Loss Function . . . 13
8.2 Curriculum Learning . . . 14
8.3 Hyperparameters . . . 14
8.4 Training Time . . . 14

9 Results **15**
9.1 Soft-Bit Bridge Version 15
9.2 Pure Neural CalcParent Version 16
9.3 Comparison with Previous Approaches 16
9.4 Parameter Count Breakdown 16

10 Complexity Analysis **17**
10.1 Per-Operation Complexity 17
10.2 Total Decoder Complexity 17
10.3 Wall-Clock Comparison 17

11 Conclusions and Future Work **17**
11.1 Summary of Achievements 17
11.2 Future Work 18

1 Introduction

Polar codes, introduced by Arıkan in 2009, provably achieve the symmetric capacity of any binary-input discrete memoryless channel under successive cancellation (SC) decoding. Their extension to the two-user multiple access channel (MAC) was established through the work of Abbe and Telatar, who showed that polar codes can achieve any rate pair in the MAC capacity region. The key insight is that the two users' codewords can be decoded jointly by treating the MAC as a set of coupled virtual channels that polarize under the Kronecker construction.

For the MAC setting, the decoding path—the order in which user U and user V bits are decoded—determines the achievable rate region. *Monotone chain paths*, parameterized by an index $i \in \{0, 1, \ldots, N\}$, trace out the entire MAC capacity region. Class A paths ($i = 0$) decode all U bits first, then all V bits. Class C paths ($i = N$) decode all V bits first, then all U bits. Class B paths ($0 < i < N$), which are required to achieve rate pairs in the interior of the capacity region, interleave U and V bit decisions.

The computational graph framework of Ren et al. (2025) provides an $O(N \log N)$ SC decoder for *any* monotone chain path. It represents the decoder state as a binary tree of probability tensors and defines three operations: `calcLeft` (top-down left child computation via circular convolution), `calcRight` (top-down right child computation via normalized product), and `calcParent` (bottom-up parent reconstruction). For Class A and C paths, only `calcLeft` and `calcRight` are needed, as the decoder traverses the tree in a single depth-first sweep. For Class B paths, however, the decoder must jump between subtrees corresponding to different users, requiring `calcParent` to propagate information upward before descending into a different subtree.

While the analytical computational graph decoder achieves $O(N \log N)$ complexity for memoryless channels, each tree operation involves manipulating $|\mathcal{S}| \times |\mathcal{S}|$ probability tensors (where $|\mathcal{S}|$ is the channel state space size), and the circular convolution in particular has complexity $O(|\mathcal{S}|^3)$. For channels with memory—such as intersymbol interference (ISI) channels or Gilbert-Elliott channels—$|\mathcal{S}|$ can be large, making the decoder computationally prohibitive.

In this work, we replace the probability tensor manipulations with neural network modules that operate on fixed-dimensional learned embeddings. Each tree operation becomes an MLP forward pass with complexity $O(md)$, where m is the hidden layer width and d is the embedding dimension. The total decoder complexity is $O(md \cdot N \log N)$, independent of the channel model. We present two architectural variants:

1. **Soft-Bit Bridge** (Section 5): The `calcParent` operation maps embeddings to log-probabilities, applies the exact analytical circular convolution, and maps back. This hybrid approach ensures correctness of the hardest operation while keeping `calcLeft` and `calcRight` fully neural. Total: 27,764 parameters.

2. **Pure Neural CalcParent** (Section 6): A GRU-gated residual network replaces `calcParent` entirely, trained via knowledge distillation from the Soft-Bit Bridge teacher. Total: 38,500 parameters.

Both variants are trained with teacher forcing using 4-class cross-entropy loss and employ curriculum learning across code lengths to scale from $N = 8$ to $N = 1024$.

2 Background: Polar Codes for the MAC

2.1 Two-User MAC Channel Model

Consider a two-user binary-input MAC where users U and V transmit binary codewords $\boldsymbol{x} = (x_1, \ldots, x_N)$ and $\boldsymbol{y} = (y_1, \ldots, y_N)$ respectively, with $x_i, y_i \in \{0, 1\}$. The receiver observes the

channel output

$$z_i = f(x_i, y_i), \quad i = 1, \ldots, N. \tag{1}$$

For the **Binary Erasure MAC (BEMAC)**, the channel is defined by addition over the integers:

$$z_i = x_i + y_i \in \{0, 1, 2\}, \tag{2}$$

so the channel output alphabet has size $|\mathcal{Z}| = 3$. When $z_i = 0$, both users sent 0; when $z_i = 2$, both sent 1; when $z_i = 1$, exactly one user sent 1 but the receiver does not know which.

For the **Gaussian MAC (GMAC)**, the channel output is:

$$z_i = (1 - 2x_i) + (1 - 2y_i) + w_i, \quad w_i \sim \mathcal{N}(0, \sigma^2), \tag{3}$$

where the binary inputs are mapped to ± 1 via BPSK modulation.

2.2 Polar Encoding

Both users employ the same polar encoder. For a code of length $N = 2^n$, the encoding operation is:

$$\boldsymbol{x} = \boldsymbol{u} \cdot G_N, \quad G_N = B_N \cdot F^{\otimes n}, \tag{4}$$

where $\boldsymbol{u} = (u_1, \ldots, u_N)$ is the input vector (containing both information and frozen bits), B_N is the $N \times N$ bit-reversal permutation matrix, and

$$F = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}, \quad F^{\otimes n} = \underbrace{F \otimes F \otimes \cdots \otimes F}_{n \text{ times}} \tag{5}$$

is the n-fold Kronecker product of the basic polarization kernel. The encoding applies a bit-reversal permutation followed by n stages of XOR butterfly operations. User V encodes identically: $\boldsymbol{y} = \boldsymbol{v} \cdot G_N$.

2.3 Monotone Chain Paths and the Capacity Region

The MAC capacity region for the BEMAC is the set of rate pairs (R_u, R_v) satisfying:

$$R_u \leq H(X|Y, Z) = 1, \tag{6}$$

$$R_v \leq H(Y|X, Z) = 1, \tag{7}$$

$$R_u + R_v \leq H(X, Y|Z) \approx 1.5. \tag{8}$$

To achieve any rate pair in this region, we must choose an appropriate *decoding path*—the order in which the $2N$ bit decisions (N from each user) are made. A **monotone chain path** is parameterized by an index $i \in \{0, 1, \ldots, N\}$ and is defined by the binary path vector:

$$\boldsymbol{b} = (\underbrace{0, \ldots, 0}_{i \text{ U-steps}}, \underbrace{1, \ldots, 1}_{N \text{ V-steps}}, \underbrace{0, \ldots, 0}_{N-i \text{ U-steps}}). \tag{9}$$

Here $b_t = 0$ indicates a U-step (decode one bit of user U) and $b_t = 1$ indicates a V-step (decode one bit of user V). The three classes are:

- **Class A** ($i = 0$): All V bits decoded first, then all U bits. Achieves the corner point $(R_u^{\max}, R_v^{\min})$.
- **Class C** ($i = N$): All U bits decoded first, then all V bits. Achieves the corner point $(R_u^{\min}, R_v^{\max})$.
- **Class B** ($0 < i < N$): Interleaved decoding. Achieves rate pairs in the interior of the capacity region. The typical choice is $i = N/2$.

2.4 The Computational Graph Decoder

The computational graph framework of Ren et al. (2025) represents the decoder state as a complete binary tree with $N-1$ internal vertices and N leaves. Each edge in the tree carries a probability tensor of shape $(|\mathcal{S}_u|, |\mathcal{S}_v|)$, representing the joint distribution over the two users' bit values. For the BEMAC, these are 2×2 tensors.

Indexing. Edges are numbered $1, 2, \ldots, 2N-1$. Edge 1 is the root edge. Edges $N, N+1, \ldots, 2N-1$ are the leaf edges. Vertex β (for $\beta \in \{1, \ldots, N-1\}$) has parent edge β, left child edge 2β, and right child edge $2\beta+1$.

Initialization. The root edge (edge 1) is initialized with the channel observation log-likelihoods, after applying the bit-reversal permutation:

$$\text{root}[t] = \log W(z_{\pi(t)} \mid u, v), \quad t = 1, \ldots, N, \tag{10}$$

where π is the bit-reversal permutation of $\{1, \ldots, N\}$.

The three core operations, defined for vertex β with parent edge of length 2ℓ and child edges of length ℓ, are:

CalcLeft (Equation 13 of Ren et al.). Computes the left child edge from the parent edge and right child edge. For each position $i = 1, \ldots, \ell$:

$$\text{left}[i] = \text{parent}[i] \circledast (\text{parent}[i+\ell] \odot \text{right}[i]), \tag{11}$$

where $\circledast$ denotes the circular convolution and $\odot$ denotes the normalized (element-wise) product on 2×2 log-probability tensors.

CalcRight (Equation 14 of Ren et al.). Computes the right child edge from the parent edge and left child edge:

$$\text{right}[i] = \text{parent}[i+\ell] \odot (\text{left}[i] \circledast \text{parent}[i]). \tag{12}$$

CalcParent (Equation 15 of Ren et al.). Reconstructs the parent edge from both child edges (bottom-up propagation):

$$\text{parent}[i] = \text{left}[i] \circledast \text{right}[i], \quad \text{parent}[i+\ell] = \text{right}[i]. \tag{13}$$

Circular Convolution. The circular convolution $C = A \circledast B$ on 2×2 log-probability tensors is defined as:

$$C[a, b] = \log \sum_{a', b'} \exp\big(A[a \oplus a', b \oplus b'] + B[a', b']\big), \tag{14}$$

where $\oplus$ denotes XOR (addition modulo 2). This yields four terms per output entry, computed via `logsumexp` for numerical stability.

Normalized Product. The normalized product $C = A \odot B$ is element-wise addition in log-space followed by normalization:

$$C[a, b] = A[a, b] + B[a, b] - \log \sum_{a', b'} \exp\big(A[a', b'] + B[a', b']\big). \tag{15}$$

3 The CalcParent Problem

3.1 Why Class B Needs CalcParent

For Class A and Class C paths, the decoder performs a single depth-first traversal of the binary tree. At each vertex, it computes `calcLeft` to descend to the left subtree, processes all leaves there, then computes `calcRight` to descend to the right subtree. The decoder never needs to propagate information *upward*—it only descends.

Class B paths break this pattern. After decoding i U-bits (which involves visiting certain leaves in the left and right subtrees), the decoder switches to decoding N V-bits, then returns to decode the remaining $N - i$ U-bits. This interleaving means the decoder must frequently jump between subtrees that are not adjacent in the tree. To navigate from one subtree to another, the decoder must first climb *up* the tree (using `calcParent` at each vertex) to reach a common ancestor, then descend *down* into the target subtree (using `calcLeft` or `calcRight`).

3.2 Frequency of CalcParent Calls

The number of `calcParent` calls depends on the path and the code length. For the standard Class B path with $i = N/2$, the decoder makes approximately $2N \log_2 N$ total tree operations, of which roughly one-third are `calcParent` calls. At $N = 1024$, this amounts to approximately 2,035 `calcParent` invocations per codeword—comparable in frequency to `calcLeft` and `calcRight` calls. This makes `calcParent` a performance-critical operation that cannot be avoided.

3.3 Why Naive Neural Approaches Fail

Before arriving at the architectures presented in this work, we explored eight distinct approaches over two development sessions, all of which failed for Class B decoding:

1. **DFS Neural Polar Decoder (NPD)**: A recursive top-down decoder that follows a depth-first traversal. Works for Class A/C but fundamentally cannot handle Class B paths because it has no mechanism for bottom-up information propagation.

2. **Bidirectional DFS**: Attempted to run two DFS passes (one for each user) and merge results. Failed because the DFS ordering does not match the Class B path ordering, leading to frozen set mismatches (BLER ≈ 1.0).

3. **Multi-pass with Conditioner**: A 3-pass architecture where an attention-based Conditioner injects codeword decisions between passes. Matched SC for Class C but failed for Class B due to train-test mismatch (Conditioner trained with true codewords, tested with noisy estimates).

4. **Iterative single-user decoder**: Alternates between decoding U (treating V as noise) and decoding V (treating U as noise). Hard decision errors in the first pass propagate catastrophically (BLER = 0.81 vs. SC = 0.056).

5. **Transformer autoregressive**: A sequence-to-sequence model following the Class B path. Matched SC at $N = 8$ (ratio 1.01) but failed at $N \geq 16$ due to autoregressive exposure bias (BLER = 0.75 vs. SC = 0.013).

6. **Naive MLP CalcParent**: Directly mapping $\mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R}^{2d}$ with an MLP. Failed due to the compression bottleneck: the circular convolution involves a 4-way logsumexp that is difficult to approximate, and small errors cascade through the tree.

The common failure mode is that Class B decoding requires *bidirectional* information flow through the tree, and any architecture that only supports top-down flow will fail. The computational graph framework provides the correct scheduling; the challenge is making each operation learnable and differentiable.

4 Architecture: Neural Computational Graph Decoder

The Neural Computational Graph (NCG) decoder retains the exact tree structure, navigation logic, and scheduling of the analytical computational graph decoder. The key modification is the *data representation*: instead of storing 2×2 log-probability tensors on each tree edge, we store learned d-dimensional embeddings. Each tree operation is replaced by a weight-shared neural network module.

4.1 Overall Architecture

The decoder maintains an array of edge data:

$$\texttt{edge_data}[\beta] \in \mathbb{R}^{B \times L_\beta \times d}, \quad \beta = 1, \ldots, 2N-1, \tag{16}$$

where B is the batch size, $L_\beta = N/2^{\lfloor \log_2 \beta \rfloor}$ is the edge length at the level of edge β, and $d = 16$ is the embedding dimension. The root edge ($\beta = 1$) has length N; leaf edges ($\beta \geq N$) have length 1.

The tree navigation functions—`stepTo`, `getPath`, and `stepOne`—are identical to those in the analytical decoder. The neural modules replace only the data-transforming operations within each step.

4.2 Neural Modules

4.2.1 EmbeddingZ (48 parameters)

The channel output embedding maps discrete channel observations to the latent space:

$$\texttt{EmbeddingZ} : \{0, 1, 2\} \to \mathbb{R}^d, \quad d = 16. \tag{17}$$

This is implemented as a standard `nn.Embedding(3, 16)` lookup table with $3 \times 16 = 48$ learnable parameters. For the BEMAC channel with $z \in \{0, 1, 2\}$, each channel output symbol is mapped to a 16-dimensional vector.

After embedding, the bit-reversal permutation is applied to align the channel observations with the computational graph's interleaved tree structure:

$$\text{root}[t] = \texttt{EmbeddingZ}(z_{\pi(t)}), \quad t = 1, \ldots, N, \tag{18}$$

where π is the bit-reversal permutation of order $n = \log_2 N$.

4.2.2 NeuralCalcLeft (8,336 parameters)

The neural `calcLeft` replaces the analytical operation of Equation (11). At vertex β with parent edge of length 2ℓ, it computes the left child edge from the parent and right child:

$$\text{left}[i] = \text{MLP}_{\text{left}}\big(\text{parent}[i] \,\|\, \text{parent}[i+\ell] \,\|\, \text{right}[i]\big), \tag{19}$$

where $\|$ denotes concatenation. The input dimension is $3d = 48$, and the architecture is:

$$\mathbb{R}^{48} \xrightarrow{\text{Linear}} \mathbb{R}^{64} \xrightarrow{\text{ELU}} \mathbb{R}^{64} \xrightarrow{\text{Linear}} \mathbb{R}^{64} \xrightarrow{\text{ELU}} \mathbb{R}^{64} \xrightarrow{\text{Linear}} \mathbb{R}^{16}. \tag{20}$$

The parameter count is: $(48\times64+64)+(64\times64+64)+(64\times16+16) = 3{,}136+4{,}160+1{,}040 = 8{,}336$.

Critically, the same network weights are shared across *all* tree levels. This weight sharing is what enables the decoder to generalize across code lengths: a model trained at $N = 16$ can decode at $N = 1024$ without retraining the `calcLeft` module.

4.2.3 NeuralCalcRight (8,336 parameters)

The neural `calcRight` has the same architecture as `calcLeft` but with a separate set of weights:

$$\text{right}[i] = \text{MLP}_{\text{right}}\big(\text{parent}[i] \,\|\, \text{parent}[i+\ell] \,\|\, \text{left}[i]\big). \tag{21}$$

Input: concatenation of parent first half, parent second half, and left child embedding, yielding $3d = 48$ input features. Same MLP architecture as `calcLeft`: $\mathbb{R}^{48} \to \mathbb{R}^{64} \to \mathbb{R}^{64} \to \mathbb{R}^{16}$ with ELU activations, totalling 8,336 parameters.

4.2.4 Emb2Logits (5,508 parameters)

The decision head maps a d-dimensional embedding to 4-class joint logits:

$$\texttt{Emb2Logits} : \mathbb{R}^{16} \to \mathbb{R}^{4}, \quad \text{output} = \big(\log P(u{=}0, v{=}0),\ \log P(u{=}0, v{=}1),\ \log P(u{=}1, v{=}0),\ \log P(u{=}1, v{=}1)\big). \tag{22}$$

Architecture: $\mathbb{R}^{16} \to \mathbb{R}^{64} \to \mathbb{R}^{64} \to \mathbb{R}^{4}$ with ELU activations. Parameter count: $(16 \times 64 + 64) + (64 \times 64 + 64) + (64 \times 4 + 4) = 1{,}088 + 4{,}160 + 260 = 5{,}508$.

This module serves a **dual purpose**:

1. At leaf positions, it produces the 4-class logits used for bit decisions.
2. In the Soft-Bit Bridge `calcParent`, it converts embeddings to log-probabilities for the analytical circular convolution.

4.2.5 Logits2Emb (5,520 parameters)

The inverse mapping from log-probability space back to the latent embedding space:

$$\texttt{Logits2Emb} : \mathbb{R}^{4} \to \mathbb{R}^{16}. \tag{23}$$

Architecture: $\mathbb{R}^{4} \to \mathbb{R}^{64} \to \mathbb{R}^{64} \to \mathbb{R}^{16}$ with ELU activations. Parameter count: $(4 \times 64 + 64) + (64 \times 64 + 64) + (64 \times 16 + 16) = 320 + 4{,}160 + 1{,}040 = 5{,}520$.

This module is used in the Soft-Bit Bridge (to re-embed the circular convolution output) and in the leaf state update (to convert partially deterministic log-probabilities to embeddings).

4.2.6 No-Info Embedding (16 parameters)

A single learnable parameter vector $\boldsymbol{e}_{\varnothing} \in \mathbb{R}^{16}$ that initializes all internal tree edges to a "no information" state at the beginning of decoding:

$$\texttt{edge_data}[\beta] = \boldsymbol{e}_{\varnothing}, \quad \forall\, \beta \in \{2, \ldots, 2N-1\}. \tag{24}$$

This vector is initialized with small random values ($\sim \mathcal{N}(0, 0.01)$) and learned end-to-end. It represents the prior state of an edge before any channel information has been propagated to it.

4.2.7 NeuralCalcParent — Pure Neural Version (10,736 parameters)

The pure neural `calcParent` module uses a GRU-style gated residual architecture. Given the left child embedding $\boldsymbol{\ell} \in \mathbb{R}^d$ and right child embedding $\boldsymbol{r} \in \mathbb{R}^d$:

$$\boldsymbol{c} = [\boldsymbol{\ell} \,\|\, \boldsymbol{r}] \in \mathbb{R}^{2d}, \tag{25}$$

$$\boldsymbol{g} = \sigma\big(\text{GateNet}(\boldsymbol{c})\big) \in [0,1]^d, \tag{26}$$

$$\hat{\boldsymbol{p}} = \text{CandidateNet}(\boldsymbol{c}) \in \mathbb{R}^d, \tag{27}$$

$$\text{parent_first}[i] = \boldsymbol{g} \odot \hat{\boldsymbol{p}} + (1 - \boldsymbol{g}) \odot \frac{\boldsymbol{\ell} + \boldsymbol{r}}{2}, \tag{28}$$

$$\text{parent_second}[i] = W_s \cdot \boldsymbol{r} + \boldsymbol{b}_s. \tag{29}$$

The **GateNet** is: $\mathbb{R}^{32} \xrightarrow{\text{Linear}} \mathbb{R}^{64} \xrightarrow{\text{ELU}} \mathbb{R}^{64} \xrightarrow{\text{Linear}} \mathbb{R}^{16} \xrightarrow{\sigma}$. Parameter count: $(32 \times 64 + 64) + (64 \times 16 + 16) = 2{,}112 + 1{,}040 = 3{,}152$.

The **CandidateNet** is: $\mathbb{R}^{32} \to \mathbb{R}^{64} \to \mathbb{R}^{64} \to \mathbb{R}^{16}$ with ELU activations, totalling $2{,}112 + 4{,}160 + 1{,}040 = 7{,}312$ parameters.

The **parent_second_nn** is a single linear layer $\mathbb{R}^{16} \to \mathbb{R}^{16}$: $16 \times 16 + 16 = 272$ parameters.

Total NeuralCalcParent: $3{,}152 + 7{,}312 + 272 = 10{,}736$ parameters.

The residual term $(\boldsymbol{\ell} + \boldsymbol{r})/2$ in Equation (28) provides a stable initialization: when the gate is 0, the output is simply the average of the children, which is a reasonable starting point. The gate learns to selectively override this average with the learned candidate representation.

5 The Soft-Bit Bridge

5.1 Architecture

The Soft-Bit Bridge is a hybrid approach to `calcParent` that keeps the analytically hardest part—the circular convolution—exact, while learning the simpler embedding-to-probability and probability-to-embedding mappings. The pipeline at vertex β is:

1. **Embedding → Log-probabilities**: Apply the shared `Emb2Logits` head followed by temperature-scaled log-softmax:

$$\boldsymbol{p}_L = \log\text{softmax}\big(\texttt{Emb2Logits}(\text{left})/\tau\big) \in \mathbb{R}^{B \times \ell \times 4}, \tag{30}$$

$$\boldsymbol{p}_R = \log\text{softmax}\big(\texttt{Emb2Logits}(\text{right})/\tau\big) \in \mathbb{R}^{B \times \ell \times 4}. \tag{31}$$

2. **Reshape to tensors**: Reshape the 4-dimensional log-probability vectors into 2×2 tensors:

$$\boldsymbol{P}_L, \boldsymbol{P}_R \in \mathbb{R}^{B \times \ell \times 2 \times 2}. \tag{32}$$

3. **Analytical circular convolution**: Compute the exact circular convolution (Equation (14)) using `torch.logsumexp`, which provides exact gradients:

$$\boldsymbol{P}_{\text{parent_first}} = \boldsymbol{P}_L \circledast \boldsymbol{P}_R. \tag{33}$$

4. **Log-probabilities → Embedding**: Re-embed both halves using `Logits2Emb`:

$$\text{parent_first_emb} = \texttt{Logits2Emb}\big(\boldsymbol{P}_{\text{parent_first}}\big), \tag{34}$$

$$\text{parent_second_emb} = \texttt{Logits2Emb}\big(\boldsymbol{p}_R\big). \tag{35}$$

5. **Concatenate**: The parent edge is formed by concatenating both halves:

$$\texttt{edge_data}[\beta] = [\text{parent_first_emb} \,\|\, \text{parent_second_emb}]. \tag{36}$$

5.2 Why It Works

The key insight is that the circular convolution—the most mathematically complex of the three tree operations—is the one kept analytical. The learned components (`Emb2Logits` and `Logits2Emb`) only need to learn relatively simple mappings between the latent space and the probability simplex.

The `Emb2Logits` module is shared with the leaf decision head, which means it is trained with direct supervision (cross-entropy loss at every leaf position). This shared training ensures that the embedding-to-probability mapping is well-calibrated, which in turn ensures that the analytical circular convolution receives valid inputs.

5.3 Differentiability

The entire Soft-Bit Bridge is differentiable. The circular convolution involves `logsumexp` operations, which have well-defined gradients:

$$\frac{\partial}{\partial A_{ij}} \log \sum_k \exp(f_k(A)) = \frac{\sum_k \exp(f_k(A)) \cdot f'_k(A)}{\sum_k \exp(f_k(A))}, \tag{37}$$

implemented automatically by PyTorch's autograd. This allows end-to-end training through the Soft-Bit Bridge without any approximation.

5.4 Temperature Parameter

The temperature τ in the log-softmax controls the sharpness of the probability distributions passed to the circular convolution. We tested $\tau \in \{1.0, 1.5, 2.0\}$ and found that $\tau = 1.0$ (no temperature scaling) yields optimal results. Higher temperatures soften the distributions, which hurts the circular convolution's ability to discriminate between different bit patterns.

5.5 Limitation

The Soft-Bit Bridge retains the $O(|\mathcal{S}|^3)$ complexity of the analytical circular convolution within `calcParent`. For memoryless channels with small alphabets (like the BEMAC with $|\mathcal{S}| = 2$), this is negligible. However, for channels with memory where the state space can be large, this becomes the computational bottleneck. This limitation motivates the pure neural `calcParent` of Section 6.

6 The Pure Neural CalcParent

6.1 Motivation

The Soft-Bit Bridge successfully solves the `calcParent` problem for memoryless channels, but its reliance on the analytical circular convolution means it inherits the $O(|\mathcal{S}|^3)$ complexity for channels with memory. For an ISI channel with memory length L, the state space grows as $|\mathcal{S}| = 2^L$, making the convolution $O(8^L)$—exponential in the memory length.

A fully neural `calcParent` with fixed architecture has complexity $O(md)$ regardless of the channel model, where $m = 64$ is the hidden dimension and $d = 16$ is the embedding dimension. This makes the total decoder complexity $O(md \cdot N \log N)$ for *any* channel.

6.2 GRU-Gated Architecture

The pure neural `calcParent` module (`NeuralCalcParent`) uses the gated residual structure detailed in Equations (26)–(29). The design choices are:

- **Residual baseline**: The term $(\boldsymbol{\ell} + \boldsymbol{r})/2$ ensures that even an untrained model produces a reasonable parent embedding (the average of its children). This stabilizes early training.
- **Sigmoid gate**: The gate $\boldsymbol{g} \in [0, 1]^d$ learns per-dimension which features should come from the candidate network versus the residual. Early in training, the gate stays near 0.5, gradually specializing.
- **Deeper candidate**: The candidate network has 2 hidden layers (vs. 1 for the gate), giving it more capacity to learn the nonlinear aspects of the circular convolution.
- **Linear parent_second**: The second half of the parent is a simple linear transform of the right child, mirroring the analytical structure where $\text{parent}[i + \ell] = \text{right}[i]$ (up to re-embedding).

6.3 Knowledge Distillation Training

Direct end-to-end training of the pure neural `calcParent` fails: the model converges to a sub-optimal local minimum where it ignores the gate and relies entirely on the residual. Knowledge distillation from the pre-trained Soft-Bit Bridge decoder provides the necessary training signal.

The distillation training proceeds in three phases:

Phase A (4,000 iterations, $\alpha = 1.0$). Only the `calcParent` parameters are trained. At each `calcParent` call, both the neural module and the analytical teacher compute the parent embedding. The loss is:

$$\mathcal{L}_A = \mathcal{L}_{\text{CE}} + \alpha \cdot \frac{1}{K} \sum_{k=1}^{K} \|\boldsymbol{e}_{\text{student}}^{(k)} - \boldsymbol{e}_{\text{teacher}}^{(k)}\|_2^2, \tag{38}$$

where K is the number of `calcParent` calls per codeword, $\boldsymbol{e}_{\text{student}}$ is the neural output, and $\boldsymbol{e}_{\text{teacher}}$ is the Soft-Bit Bridge output (detached from the computation graph). All other parameters (`calcLeft`, `calcRight`, `Emb2Logits`, etc.) are frozen.

Phase B (4,000 iterations, $\alpha : 1.0 \to 0.0$). Still training only `calcParent` parameters, but the distillation weight α linearly decays from 1.0 to 0.0. This gradually shifts the training signal from imitating the teacher to optimizing the cross-entropy loss directly.

Phase C (4,000 iterations, $\alpha = 0.0$). All parameters are unfrozen and fine-tuned jointly at reduced learning rates (CalcParent: 10^{-4}, others: 5×10^{-5}). No teacher signal. This allows the entire network to co-adapt to the neural `calcParent`.

Critical discovery. During distillation (Phases A and B), the pre-trained parameters *must be frozen*. If they are allowed to train simultaneously, the `Emb2Logits` module (shared between the teacher's Soft-Bit Bridge and the decision head) drifts, causing the teacher signal to become inconsistent and training to diverge.

Verification. At inference time, the pure neural decoder makes zero analytical `calcParent` calls. For $N = 16$ with path index $i = 8$, the decoder makes 345 neural `calcParent` calls and 0 teacher calls per codeword, confirming that all analytical operations have been eliminated.

7 One Decoding Step

We now walk through the six sub-steps that occur when the decoder visits a single leaf position. Let the current position in the path vector be step t with $b_t = \gamma$ (0 for U-step, 1 for V-step), and let the target leaf edge be e_{leaf} at vertex $v_{\text{target}} = e_{\text{leaf}} \gg 1$.

7.1 Step 1: Navigate to Target Leaf

The decoder maintains a "decoding head" position in the tree, initially at vertex 1 (the root). To reach the target vertex, it calls `stepTo(current, target)`, which:

(a) Computes the path between `current` and `target` via `getPath`: climb up from `current` to the lowest common ancestor, then descend to `target`.

(b) For each step in the path, calls `stepOne`:

- **Going up** (current → parent): invokes `calcParent` on the current vertex.
- **Going down to left child**: invokes `calcLeft` on the current vertex.
- **Going down to right child**: invokes `calcRight` on the current vertex.

7.2 Step 2: Save Bottom-Up Embedding

Before computing the top-down message, the decoder saves the current state of the leaf edge, which represents the bottom-up ("getAsParent") information accumulated from previous leaf visits:

$$\boldsymbol{t}_{\text{temp}} = \texttt{edge_data}[e_{\text{leaf}}][:, 0] \in \mathbb{R}^{B\times d}. \tag{39}$$

The `.clone()` ensures that this tensor is not modified by subsequent operations.

7.3 Step 3: Compute Top-Down Message

The top-down message is computed by applying `calcLeft` (if e_{leaf} is a left child, i.e., e_{leaf} is even) or `calcRight` (if e_{leaf} is a right child, i.e., e_{leaf} is odd) at the target vertex:

$$\boldsymbol{t}_{\text{td}} = \texttt{edge_data}[e_{\text{leaf}}][:, 0] \in \mathbb{R}^{B\times d}. \tag{40}$$

This overwrites the leaf edge data with the top-down information from the parent.

7.4 Step 4: Combine Top-Down and Bottom-Up

The combined embedding merges the top-down channel information with the bottom-up prior information via simple addition:

$$\boldsymbol{t}_{\text{combined}} = \boldsymbol{t}_{\text{td}} + \boldsymbol{t}_{\text{temp}} \in \mathbb{R}^{B\times d}. \tag{41}$$

This additive combination was found to work as well as a learned combiner MLP (`use_combine_nn=True` option) while being simpler and having zero additional parameters.

7.5 Step 5: Make Decision

The combined embedding is passed through the `Emb2Logits` decision head to obtain 4-class joint logits:

$$\boldsymbol{\ell} = \texttt{Emb2Logits}(\boldsymbol{t}_{\text{combined}}) = \big(\ell_{00},\, \ell_{01},\, \ell_{10},\, \ell_{11}\big) \in \mathbb{R}^{B\times 4}. \tag{42}$$

The decision depends on whether this is a U-step or V-step:

- **U-step** ($\gamma = 0$): Marginalize over v to get $P(u)$:

$$P(u = 0) = \text{logsumexp}(\ell_{00}, \ell_{01}), \tag{43}$$
$$P(u = 1) = \text{logsumexp}(\ell_{10}, \ell_{11}). \tag{44}$$

  Decide $\hat{u} = \mathbf{1}[P(u = 1) > P(u = 0)]$.

- **V-step** ($\gamma = 1$): Marginalize over u to get $P(v)$:

$$P(v = 0) = \text{logsumexp}(\ell_{00}, \ell_{10}), \tag{45}$$
$$P(v = 1) = \text{logsumexp}(\ell_{01}, \ell_{11}). \tag{46}$$

  Decide $\hat{v} = \mathbf{1}[P(v = 1) > P(v = 0)]$.

If the current position is in the frozen set, the decision is overridden with the known frozen value.

7.6 Step 6: Update Leaf to Partially Deterministic State

After the decision, the leaf embedding is updated to reflect the now-known bit value. This is the neural equivalent of Equation 16 in Ren et al. (2025). A 4-class log-probability vector is constructed:

- If both u and v are known at this position: set the known (u, v) entry to $\log 1 = 0$ and all others to -30 (effectively $-\infty$).
- If only u is known: set $P(u, v{=}0) = P(u, v{=}1) = \log 0.5$ (uniform over v), others to -30.
- If only v is known: set $P(u{=}0, v) = P(u{=}1, v) = \log 0.5$ (uniform over u), others to -30.

This log-probability vector is then mapped back to the latent space:

$$\texttt{edge_data}[e_{\text{leaf}}] = \texttt{Logits2Emb}(\boldsymbol{p}_{\text{partial}}).\text{unsqueeze}(1). \tag{47}$$

Importantly, we store the *partially deterministic* embedding, not the combined top-down message. Storing the combined message would double-count the channel information when the leaf is revisited in a future `calcParent` call.

8 Training

8.1 Teacher Forcing and Loss Function

Training uses teacher forcing: at each leaf position, the true bit values from the ground truth codewords are used for the decision (rather than the model's own predictions). This eliminates exposure bias during training and allows the loss to be computed at every leaf position independently.

The training convention uses **all-info** (no frozen set): every position is treated as an information bit. This maximizes the number of supervised positions per training example and avoids the need to design frozen sets for training.

The loss function is the 4-class cross-entropy summed over all $2N$ leaf positions:

$$\mathcal{L} = \frac{1}{2N} \sum_{t=1}^{2N} \text{CE}\big(\boldsymbol{\ell}_t,\ u_t \cdot 2 + v_t\big), \tag{48}$$

where $\boldsymbol{\ell}_t$ is the 4-class logit vector at step t, and $u_t \cdot 2 + v_t \in \{0, 1, 2, 3\}$ is the target class index.

8.2 Curriculum Learning

Curriculum learning is essential for scaling the decoder to large code lengths. Training from scratch succeeds only for $N \leq 16$; for $N \geq 32$, the loss gets stuck at approximately 1.04 (which corresponds to random 4-class guessing with slight bias, far from the optimal $\sim$0.173).

The curriculum proceeds as follows:

1. Train from scratch at $N = 8$ until convergence ($\sim$20K iterations).
2. Train from scratch at $N = 16$ until convergence ($\sim$20K iterations).
3. Load the $N = 16$ checkpoint and fine-tune at $N = 32$ ($\sim$20K iterations).
4. Load the $N = 32$ checkpoint and fine-tune at $N = 64$ ($\sim$15K iterations).
5. Continue: $N = 128 \to 256 \to 512 \to 1024$.

At every code length, the training loss converges to approximately $\mathcal{L} \approx 0.173$, which represents the information-theoretic minimum for the BEMAC channel (the conditional entropy $H(U, V \mid Z)$ averaged over positions).

The reason curriculum learning works is weight sharing: the `calcLeft`, `calcRight`, and `calcParent` modules are applied identically at every tree level. A model trained at $N = 16$ (4 levels) has already learned these operations well; fine-tuning at $N = 32$ (5 levels) only requires adapting to the new root embedding and slightly different tree statistics, not relearning the operations from scratch.

8.3 Hyperparameters

Table 1: Training hyperparameters for each code length.

N	Batch size	Iterations	Learning rate	Schedule	Init
8	64	20,000	10^{-3}	Cosine ($\eta_{\min} = 5 \times 10^{-5}$)	Scratch
16	64	20,000	10^{-3}	Cosine ($\eta_{\min} = 5 \times 10^{-5}$)	Scratch
32	64	20,000	5×10^{-4}	Cosine ($\eta_{\min} = 2.5 \times 10^{-5}$)	N=16 ckpt
64	32	15,000	3×10^{-4}	Cosine ($\eta_{\min} = 1.5 \times 10^{-5}$)	N=32 ckpt
128	16	15,000	2×10^{-4}	Cosine ($\eta_{\min} = 10^{-5}$)	N=64 ckpt
256	16	10,000	10^{-4}	Cosine ($\eta_{\min} = 5 \times 10^{-6}$)	N=128 ckpt
512	8	10,000	10^{-4}	Cosine ($\eta_{\min} = 5 \times 10^{-6}$)	N=256 ckpt
1024	8	10,000	5×10^{-5}	Cosine ($\eta_{\min} = 2.5 \times 10^{-6}$)	N=512 ckpt

All experiments use the Adam optimizer with gradient clipping at norm 1.0. No weight decay is applied. The cosine annealing schedule reduces the learning rate from the initial value to $\eta_{\min}$ over the full training run.

8.4 Training Time

Training is performed on CPU (Apple M-series). Per-iteration wall-clock time scales linearly with N (due to the sequential nature of SC decoding along the path):

Table 2: Approximate training time per code length (CPU).

N	Time/iter (ms)	Total iters	Total time	Cumulative
8	15	20,000	5 min	5 min
16	35	20,000	12 min	17 min
32	80	20,000	27 min	44 min
64	180	15,000	45 min	1.5 hr
128	400	15,000	1.7 hr	3.2 hr
256	900	10,000	2.5 hr	5.7 hr
512	2000	10,000	5.6 hr	11.3 hr
1024	4500	10,000	12.5 hr	23.8 hr

The total curriculum training from $N = 8$ to $N = 1024$ takes approximately 24 hours on a single CPU core.

9 Results

All results are for the BEMAC channel ($Z = X + Y$) with Class B decoding path (path index $i = N/2$). The rate point is approximately $R_u \approx 0.5$, $R_v \approx 0.7$, which is an interior point of the MAC capacity region achievable only with Class B paths.

9.1 Soft-Bit Bridge Version

Table 3: BLER comparison: Soft-Bit Bridge NCG decoder vs. analytical SC decoder. Class B path, $R_u \approx 0.5$, $R_v \approx 0.7$.

N	k_u	k_v	NN BLER	SC BLER	Ratio (NN/SC)
8	4	6	0.0586	0.0582	1.01
16	8	11	0.0106	0.0118	**0.90**
32	16	22	0.0082	0.0096	**0.85**
64	32	45	0.0044	0.0056	**0.79**
128	64	90	0.0018	0.0034	**0.53**
512	256	358	0.0030	0.0040	**0.75**
1024	512	717	0.0040	0.0080	**0.50**

Key observations:

- At $N = 8$, the NN matches SC almost exactly (ratio 1.01), validating the architecture.
- For $N \geq 16$, the NN consistently *outperforms* analytical SC, with the gap widening at larger N.
- At $N = 1024$, the NN achieves a $2\times$ BLER improvement (ratio 0.50).
- The improvement is not due to overfitting: the NN is trained with all-info (no frozen set) and evaluated with the proper Class B frozen set designed via Monte Carlo methods.

9.2 Pure Neural CalcParent Version

Table 4: BLER comparison: Pure Neural CalcParent NCG decoder vs. analytical SC. Class B path, $N = 16$, $k_u = 8$, $k_v = 11$.

Phase	NN BLER	SC BLER
Phase A (distill, CalcParent only, $\alpha = 1.0$)	0.013	0.010
Phase B (decay $\alpha : 1.0 \to 0.0$, CalcParent only)	0.013	0.010
Phase C (finetune all, $\alpha = 0$)	0.013	0.010
Best NN / SC ratio	1.30	

The pure neural version achieves a ratio of 1.30 at $N = 16$—competitive with analytical SC despite having zero analytical operations at inference time. The training loss converges to the same $\sim$0.173 value as the Soft-Bit Bridge version, indicating that the neural `calcParent` successfully learns the circular convolution.

9.3 Comparison with Previous Approaches

Table 5: Comparison of Class B decoding approaches at $N = 8$, $R_u = 0.5$, $R_v \approx 0.7$.

Approach	NN BLER	SC BLER	Ratio
DFS NPD (Class C only)	—	—	N/A (cannot decode Class B)
Iterative single-user	0.810	0.058	13.9 (catastrophic)
Multi-pass + Conditioner	0.062	0.058	1.07 (Class C only)
Transformer (351K params)	0.063	0.062	1.01 (N=8 only)
NCG Soft-Bit Bridge (27K params)	**0.059**	**0.058**	**1.01**
NCG Pure Neural (38K params)	**0.013***	**0.010***	**1.30***

* Evaluated at $N = 16$.

The NCG decoder is the only approach that: (1) successfully handles Class B paths, (2) scales to large N via curriculum learning, and (3) uses fewer parameters than the Transformer.

9.4 Parameter Count Breakdown

Table 6: Parameter counts for both decoder versions.

Module	Soft-Bit Bridge	Pure Neural
EmbeddingZ (`nn.Embedding(3, 16)`)	48	48
NeuralCalcLeft (MLP $48 \to 64 \to 64 \to 16$)	8,336	8,336
NeuralCalcRight (MLP $48 \to 64 \to 64 \to 16$)	8,336	8,336
Emb2Logits (MLP $16 \to 64 \to 64 \to 4$)	5,508	5,508
Logits2Emb (MLP $4 \to 64 \to 64 \to 16$)	5,520	5,520
no_info_emb (learnable $\mathbb{R}^{16}$)	16	16
NeuralCalcParent (GRU-gated)	—	10,464
parent_second_nn (Linear $16 \to 16$)	—	272
Total	**27,764**	**38,500**

10 Complexity Analysis

10.1 Per-Operation Complexity

Each neural tree operation is an MLP forward pass:

Table 7: Per-operation complexity comparison.

Operation	Analytical	Neural
`calcLeft`	$O(\|\mathcal{S}\|^3)$	$O(md)$
`calcRight`	$O(\|\mathcal{S}\|^2)$	$O(md)$
`calcParent` (Soft-Bit Bridge)	—	$O(\|\mathcal{S}\|^3 + md)$
`calcParent` (Pure Neural)	—	$O(md)$
Leaf decision	$O(\|\mathcal{S}\|^2)$	$O(md)$

For the BEMAC, $|\mathcal{S}| = 2$, so the analytical operations are $O(1)$. The neural operations with $m = 64$, $d = 16$ involve matrix multiplications of size 48×64, 64×64, and 64×16, totalling approximately 8,300 multiply-add operations per tree step.

10.2 Total Decoder Complexity

The computational graph decoder makes $O(N \log N)$ tree operations for any monotone chain path (Ren et al. 2025). Therefore:

$$\text{Analytical SC (memoryless):} \quad O(N \log N), \tag{49}$$

$$\text{Analytical SC (memory } |\mathcal{S}|\text{):} \quad O(|\mathcal{S}|^3 \cdot N \log N), \tag{50}$$

$$\text{Neural SC (Soft-Bit Bridge):} \quad O\big((|\mathcal{S}|^3 + md) \cdot N \log N\big), \tag{51}$$

$$\text{Neural SC (Pure Neural):} \quad O(md \cdot N \log N). \tag{52}$$

The pure neural decoder's complexity is *independent of the channel*. With $m = 64$ and $d = 16$, the constant factor is $md = 1{,}024$, which is fixed regardless of whether the channel is a simple BEMAC or a complex ISI channel with large state space.

10.3 Wall-Clock Comparison

On CPU (Apple M-series), the neural decoder is approximately 1.5× slower than the analytical SC decoder per codeword at $N = 1024$. This overhead comes from the MLP forward passes at each tree operation, which involve matrix multiplications that are more expensive than the direct 2×2 tensor manipulations of the analytical decoder.

However, the neural decoder has significant potential for GPU acceleration: all MLP operations within a single tree step can be batched across positions (the edge data at each level has shape (B, L, d), where L positions can be processed in parallel), and the batch dimension B provides additional parallelism.

11 Conclusions and Future Work

11.1 Summary of Achievements

We have presented the Neural Computational Graph (NCG) SC decoder for two-user MAC polar codes, which replaces the analytical tree operations of the Ren et al. (2025) framework with learned neural modules. The key contributions are:

1. **Soft-Bit Bridge architecture**: A hybrid `calcParent` that keeps the circular convolution exact while learning the embedding-probability mappings. At 27,764 parameters, it matches or outperforms analytical SC from $N = 8$ to $N = 1024$, achieving a $2\times$ BLER improvement at $N = 1024$.

2. **Pure Neural CalcParent**: A GRU-gated residual module trained via knowledge distillation that eliminates all analytical operations. At 38,500 parameters and $O(md \cdot N \log N)$ complexity, it is channel-independent.

3. **Curriculum learning**: A practical training strategy that scales the decoder from $N = 8$ to $N = 1024$ by fine-tuning across code lengths, exploiting the weight-shared architecture.

4. **Dual-purpose Emb2Logits**: The shared decision head / Soft-Bit Bridge input module provides a natural bridge between the latent embedding space and the probability domain.

11.2 Future Work

Channels with Memory. The primary motivation for the pure neural `calcParent` is channels where the analytical operations become prohibitively expensive. Preliminary experiments on ISI channels show that the training loss converges to the expected information-theoretic minimum, suggesting that the architecture can learn channel-dependent operations without modification. Formal evaluation on Gilbert-Elliott and ISI channels is ongoing.

Neural SCL (List Decoding). Successive cancellation list (SCL) decoding maintains L candidate paths and achieves significantly better BLER than SC. The NCG architecture naturally extends to SCL by maintaining L copies of the edge data and using the `Emb2Logits` output to rank candidates. The main challenge is differentiable path pruning.

GPU Acceleration. The current implementation processes tree operations sequentially along the decoding path. GPU acceleration could exploit: (a) batch parallelism across codewords, (b) position parallelism within each edge (positions $1, \ldots, L$ at each tree level are independent), and (c) pipelining of consecutive tree operations.

Frozen Set Co-Design. Current frozen set design uses Monte Carlo estimation with the analytical decoder. Jointly optimizing the frozen set and the neural decoder could yield rate points specifically suited to the neural decoder's strengths, potentially extending the region where the NN outperforms SC.

Extension to $K > 2$ Users. The computational graph framework generalizes to K-user MACs with probability tensors of shape 2^K. The neural modules would require 2^K-class output logits but maintain the same $O(md \cdot N \log N)$ tree structure.